#5 (A) Find the minimum `MR`, in the right branch, for current node's split coordinate.
#6 (B) Remove `MR` above from the right subtree.
#7 (C) Create a new node, to replace current, with `MR` as point, and the old branches.
#8 If, instead, there is no right child, but there is a left branch, we are in the first special case:
#9 (A) Find the minimum `ML`, in the left branch, for current node's split coordinate.
#10 (B) Remove `ML` found above from the left branch.
#11 (C) Create a new node, to replace current, using `ML` as point, and setting left child to `null`, and right child to the former left branch.
#12 If both left and right branches are null, we are at a leaf, so we can just remove it.
#13/#14 Otherwise, if current node doesn't match the point to delete, check if we need to go left or right, and recursively call remove.

Our recurrence would therefore become:

$$T_k(n) = T_k(n/2) + O(n^{1-1/k})$$

Using master theorem's third case, since $1-1/k > \log2(1)=0$, and $(n/2)1-1/k \leq n1-1/k$, we can then conclude that the amortized time required by `remove` on a balanced k-d tree is:

$$T_k(n) = O(n^{1-1/k})$$